

Rudder Analytics Round 2: Last-Night Python Cheat Sheet

For Karan | Test: Fri 9 Oct 2026, 10:30 AM onwards | On-campus | 90 min | Associate Analyst

1. What to expect (30-second read)

- **From your notice:** Round 2 is a 90-minute on-campus programming test for CS/IT candidates, held Fri 9 Oct from 10:30 AM. Slots come by email, so check inbox and Spam tonight.
- **From recent candidate reports (Glassdoor / AmbitionBox, self-reported, small sample, so treat as likely, not guaranteed):**
 - Pune, Mar 2026: *2 easy questions in 90 min.*
 - Pune: simple programs on **strings, linked lists and arrays**.
 - Others mention **reverse a string, array indexing, sorting an array**, a **SQL join** question, and 'predict the output' questions.
 - Tests run on the company's own software.
- **Bottom line:** easy-level problems. You win by being clean on basics, handling edge cases, and not freezing. You do NOT need advanced DSA.
- **Confirm at the start:** that Python is an allowed language, and how input is given (stdin vs function).

2. 90-minute game plan

Time	Do this
0-5 min	Read BOTH questions fully. Pick the easier one first.
5-40 min	Solve Q1. Write the brute-force/simple way first. Run sample input.
40-75 min	Solve Q2 the same way.

Time	Do this
75-90 min	Re-test edge cases (empty, single item, duplicates, negatives). Remove debug prints. Submit.

Rules: simple and correct beats clever. Never leave a question blank (partial test cases count). If stuck for 10 min, write the brute force and move on.

3. Python survival kit

```
# --- Input ---
n = int(input())                # one integer
a = list(map(int, input().split())) # '1 2 3' -> [1, 2, 3]
s = input().strip()             # a string line
x, y = map(int, input().split()) # two ints on one line
# input() takes NO prompt text. Do not print extra words in output.

# --- Output ---
print(*a)                       # 1 2 3 (space separated)
print(' '.join(map(str, a)))
print(a[0], end=' ')            # no newline
print(f'{x:.2f}')               # 2 decimal places

# --- Slicing ---
s[::-1]      # reverse
s[1:4]       # index 1..3
a[-1]        # last item
a[:k], a[k:] # first k, rest

# --- Handy built-ins ---
len(a); sum(a); max(a); min(a); sorted(a); sorted(a, reverse=True)
set(a) # unique items (order lost)
list(dict.fromkeys(a)) # unique, order kept
from collections import Counter
Counter('hello') # {'l': 2, 'h': 1, 'e': 1, 'o': 1}
divmod(17, 5)    # (3, 2)
for i, v in enumerate(a): ...
```

String methods: `.lower()` `.upper()` `.strip()` `.split()` `.join()` `.replace(a,b)`
`.isalpha()` `.isdigit()` `.isalnum()` `.count(x)` `.find(x)` `.startswith(x)`

Remember: `//` is integer division, `%` is remainder, `**` is power. Strings are immutable. Indexes start at 0.

4. Strings (most likely topic)

```
# 1. Reverse a string
def rev(s):
    return s[::-1]

# 2. Palindrome (ignore case and non-letters)
def is_pal(s):
    t = ''.join(c.lower() for c in s if c.isalnum())
    return t == t[::-1]

# 3. Count vowels
def vowels(s):
    return sum(1 for c in s.lower() if c in 'aeiou')

# 4. Anagram check
from collections import Counter
def anagram(a, b):
    a = a.replace(' ', '').lower()
    b = b.replace(' ', '').lower()
    return Counter(a) == Counter(b)          # or sorted(a) == sorted(b)

# 5. Character frequency
def freq(s):
    d = {}
    for c in s:
        d[c] = d.get(c, 0) + 1
    return d

# 6. First non-repeating character
def first_unique(s):
    cnt = Counter(s)
    for c in s:
        if cnt[c] == 1:
            return c
    return None

# 7. Reverse words in a sentence
def rev_words(s):
```

```

    return ' '.join(s.split()[::-1])      # 'I love Python' -> 'Python love I'

# 8. Reverse each word, keep word order
def rev_each(s):
    return ' '.join(w[::-1] for w in s.split())

# 9. Compress: 'aaabbc' -> 'a3b2c1'
def compress(s):
    if not s:
        return ''
    out, count = [], 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            out.append(s[i - 1] + str(count))
            count = 1
    out.append(s[-1] + str(count))
    return ''.join(out)

# 10. Remove duplicate characters (keep order)
def dedup(s):
    return ''.join(dict.fromkeys(s))

# 11. Longest word / word count
def longest(s):
    return max(s.split(), key=len)
word_count = len('a b c'.split())      # 3 (split() handles extra spaces)

# 12. Pangram (all 26 letters)
def pangram(s):
    return set('abcdefghijklmnopqrstuvwxyz') <= set(s.lower())

# 13. Swap case / capitalise words
'Hello'.swapcase()      # 'hELLO'
'hello world'.title()   # 'Hello World'

# 14. Most frequent character
def most_common(s):
    return Counter(s).most_common(1)[0][0]

# 15. Longest common prefix

```

```
def lcp(words):
    if not words:
        return ''
    p = words[0]
    for w in words[1:]:
        while not w.startswith(p):
            p = p[:-1]
    return p
```

5. Arrays / lists

```
# 1. Largest and second largest
def second_largest(a):
    u = sorted(set(a))
    return u[-2] if len(u) > 1 else None

# 2. Sort without sorted() (bubble sort, if they forbid built-ins)
def bubble(a):
    n = len(a)
    for i in range(n):
        for j in range(n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a

# 3. Remove duplicates (keep order)
def uniq(a):
    return list(dict.fromkeys(a))

# 4. Rotate right by k
def rotate(a, k):
    if not a:
        return a
    k %= len(a)
    return a[-k:] + a[:-k] if k else a

# 5. Two Sum: indices of two numbers adding to target
def two_sum(a, target):
    seen = {}
    for i, v in enumerate(a):
        if target - v in seen:
```

```

        return [seen[target - v], i]
    seen[v] = i
    return []

# 6. Missing number in 1..n (list has n-1 items)
def missing(a, n):
    return n * (n + 1) // 2 - sum(a)

# 7. Move zeros to end
def move_zeros(a):
    return [x for x in a if x != 0] + [0] * a.count(0)

# 8. Max subarray sum (Kadane)
def kadane(a):
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best

# 9. Merge two sorted lists
def merge(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]

# 10. Binary search (sorted list) -> index or -1
def bsearch(a, t):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == t:
            return mid
        if a[mid] < t:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

```

```

# 11. Find duplicates
def dups(a):
    c = Counter(a)
    return [k for k, v in c.items() if v > 1]

# 12. Frequency / most repeated element
def most_freq(a):
    return Counter(a).most_common(1)[0][0]

# 13. Count pairs with a given sum (simple  $O(n^2)$ )
def pairs(a, s):
    return sum(1 for i in range(len(a)) for j in range(i + 1, len(a)) if a[i] + a[j] == s)

# 14. Array indexing questions: even/odd index items, reverse in place
evens = a[0::2]    # items at index 0, 2, 4...
odds = a[1::2]
a.reverse()        # in place

# 15. Check if sorted
def is_sorted(a):
    return all(a[i] <= a[i + 1] for i in range(len(a) - 1))

```

6. Linked list (mentioned in reports, learn this block)

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def build(arr):
    head = tail = None
    for x in arr:
        n = Node(x)
        if head is None:
            head = tail = n
        else:
            tail.next = n
            tail = n
    return head

```

list -> linked list

```

def to_list(head):                                # print / traverse
    out = []
    while head:
        out.append(head.data)
        head = head.next
    return out

def reverse(head):                                # MOST ASKED
    prev = None
    cur = head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev

def middle(head):                                # slow / fast pointers
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

def insert_end(head, val):
    n = Node(val)
    if not head:
        return n
    cur = head
    while cur.next:
        cur = cur.next
    cur.next = n
    return head

```

```
def delete_value(head, val):
    dummy = Node(0)
    dummy.next = head
    cur = dummy
    while cur.next:
        if cur.next.data == val:
            cur.next = cur.next.next
            break
        cur = cur.next
    return dummy.next
```

```
def length(head):
    n = 0
    while head:
        n += 1
        head = head.next
    return n
```

```
def merge_sorted(a, b):
    dummy = tail = Node(0)
    while a and b:
        if a.data <= b.data:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

```
# Test: print(to_list(reverse(build([1, 2, 3, 4]))) -> [4, 3, 2, 1]
```

Trick: the `dummy` node avoids special-casing the head. Use it for any delete/merge question.

7. Number problems

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True

def fib(n):
    # first n terms
    a, b, out = 0, 1, []
    for _ in range(n):
        out.append(a)
        a, b = b, a + b
    return out

def fact(n):
    r = 1
    for i in range(2, n + 1):
        r *= i
    return r

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
    # lcm = a * b // gcd(a, b)

def digit_sum(n):
    return sum(int(d) for d in str(abs(n)))

def rev_num(n):
    return int(str(n)[::-1])

def is_palindrome_num(n):
    return str(n) == str(n)[::-1]

def armstrong(n):
    # 153 = 1^3 + 5^3 + 3^3
    d = str(n)
    return n == sum(int(c) ** len(d) for c in d)
```

```
def fizzbuzz(n):
    for i in range(1, n + 1):
        if i % 15 == 0: print('FizzBuzz')
        elif i % 3 == 0: print('Fizz')
        elif i % 5 == 0: print('Buzz')
        else: print(i)

def primes_upto(n):
    # all primes <= n
    return [x for x in range(2, n + 1) if is_prime(x)]

def power(b, e):
    # recursion example
    return 1 if e == 0 else b * power(b, e - 1)
```

8. Pattern printing

```
n = 4
# Right triangle      # Pyramid
for i in range(1, n + 1):
    print('*' * i)
for i in range(1, n + 1):
    print(' ' * (n - i) + '*' * (2 * i - 1))
# Number triangle: row i prints 1..i
for i in range(1, n + 1):
    print(*range(1, i + 1))
```

9. Matrix (2D list)

```
m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
transpose = [list(r) for r in zip(*m)]
row_sums = [sum(r) for r in m]
col_sums = [sum(c) for c in zip(*m)]
main_diag = sum(m[i][i] for i in range(len(m)))
# Create a blank 3x3 CORRECTLY:
grid = [[0] * 3 for _ in range(3)]    # NOT [[0]*3]*3 (rows would be shared)
```

10. SQL (reported once or twice, 10 min revision)

Sample tables: `employees(id, name, dept_id, salary)`, `departments(id, dept_name)`.

```

-- INNER JOIN (only matching rows)
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.id;

-- LEFT JOIN (all employees, even without a department)
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id;

-- GROUP BY + HAVING
SELECT dept_id, COUNT(*) AS cnt, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 2;

-- Second highest salary
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Find duplicate names
SELECT name, COUNT(*) FROM employees GROUP BY name HAVING COUNT(*) > 1;

-- Top 3 salaries
SELECT * FROM employees ORDER BY salary DESC LIMIT 3;

```

Join memory aid: INNER = intersection. LEFT = everything on the left plus matches.
 WHERE filters rows before grouping; HAVING filters after.

11. Pandas mini-kit (it is a data role, so know these)

```
import pandas as pd
df = pd.read_csv('file.csv')
df.head(); df.info(); df.describe(); df.shape
df[df['salary'] > 50000]          # filter
df[['name', 'salary']]          # select columns
df.sort_values('salary', ascending=False)
df.groupby('dept')['salary'].mean() # group + aggregate
df['salary'].fillna(df['salary'].mean()) # fill missing
df.dropna(); df.drop_duplicates()
df['bonus'] = df['salary'] * 0.1  # new column
pd.merge(df1, df2, on='id', how='left') # join
df['dept'].value_counts()
```

12. 'Predict the output' traps

```
print(7 / 2)          # 3.5    (/ is always float)
print(7 // 2)         # 3
print(-7 // 2)        # -4     (floors, not truncates)
print(0.1 + 0.2)      # 0.30000000000000004
print('ab' * 3)       # ababab
print(bool([]), bool(' ')) # False True

a = [1, 2]; b = a; b.append(3)
print(a)              # [1, 2, 3] (same object; use a.copy() for a real copy)

def f(x, lst=[]):     # mutable default argument bug
    lst.append(x)
    return lst
print(f(1), f(2))    # [1, 2] [1, 2] (same list reused!)

print('hello'[10:])  # '' (slices never raise IndexError)
# 'hello'[10] raises IndexError
s = 'abc'            # s[0] = 'x' raises TypeError (strings immutable)
print(range(3))      # range(0, 3), not a list
print(5 == 5.0, 5 is 5.0) # True False (is checks identity)
for i in range(5):
```

```

pass
print(i)          # 4 (loop variable survives)
print(sorted('bca')) # ['a', 'b', 'c'] (a list, not a string)
print(type(input())) # <class 'str'> always

```

13. OOP / theory one-liners (in case of a quick question)

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def grade(self):
        return 'A' if self.marks >= 75 else 'B'
s = Student('Karan', 80); print(s.grade())

```

- **List vs tuple:** list is mutable, tuple is not. **Set:** unique, unordered. **Dict:** key -> value, O(1) lookup.
- **Time complexity:** loop = O(n), nested loop = O(n²), sorting = O(n log n), dict/set lookup = O(1).
- **Exception handling:** `try: ... except ValueError: ... finally: ...`

14. Edge-case checklist (run through before submitting)

- Empty input (empty string / empty list)? Single element?
- All same elements? Duplicates? Negative numbers? Zero?
- Case sensitivity ('Aba' palindrome?) and spaces/punctuation?
- Output format exactly as asked (spaces vs commas, True vs true, no extra prints)?
- Did you strip debug `print()` lines and avoid prompt text inside `input()`?
- Large input? Avoid needless nested loops if n can be big.

15. Tonight's plan (about 90 minutes total)

1. **30 min:** read sections 3 to 6 once, slowly.
2. **40 min:** TYPE (do not just read) from memory: reverse string, palindrome, anagram, second largest, two-sum, Kadane, rotate list, linked-list build + reverse + middle. Run each with 2 test inputs.

3. **10 min:** skim sections 7, 10 and 12.

4. **Sleep.** A rested brain beats one extra problem.

16. Tomorrow's logistics (from the notice)

- Report **30 minutes before your allotted slot** (Round 2 starts from 10:30 AM, so check your email for your exact slot).
- Check **registered email + Spam/Junk** for slot and links. Slot changes and re-tests may not be possible.
- Carry laptop/desktop with **working webcam**, charger, and a stable internet connection.
- Open the test link in **Incognito/Private mode**. **Do not switch browsers** or click outside the exam window, as this can cancel your test.
- Stay available afterwards: Round 3 (GD and PI) schedule will be emailed this week or next.

You've got this: the reports describe easy-level questions, so stay calm, read carefully, and write simple correct code.